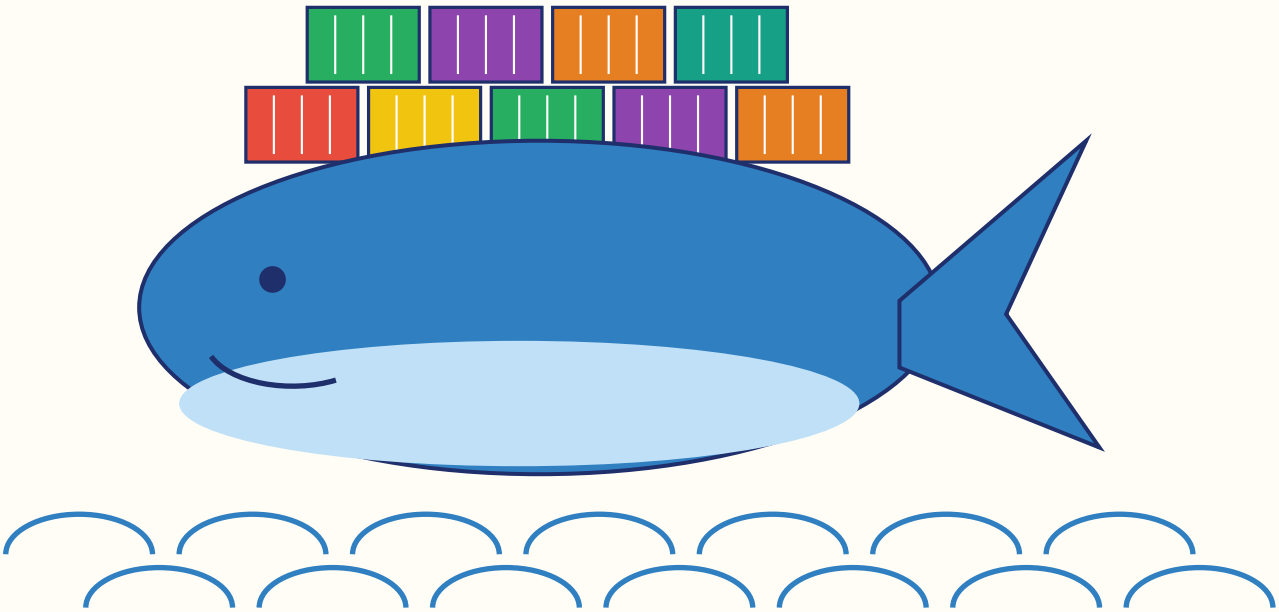
DOCKER

A Complete Beginner's Guide

concepts + examples + every command you need



What's inside:

- [ ] Containers, images and the Docker vocabulary
- [ ] Installing Docker + your first containers
- [ ] Essential commands for images and containers
- [ ] Writing a Dockerfile (Node.js + FastAPI examples)
- [ ] Volumes, networks and databases
- [ ] Docker Compose, Docker Hub, cleanup
- [ ] Troubleshooting, cheat sheet and practice tasks

Tip: type every command yourself - that's how it sticks!

Before You Start

These notes assume **zero** Docker knowledge. If you can open a terminal and type a command, you are ready. Every chapter follows the same pattern: **idea -> picture -> commands -> try it yourself**.

What you should know already

- How to open a terminal (Command Prompt / PowerShell on Windows, Terminal on Mac and Linux).
- A few basic commands like `cd` (change folder) and `ls` (list files).
- Nothing about servers or Linux - I explain as we go.

How to read these notes

- Yellow boxes with `code` are commands you type. Lines starting with `#` are comments - do not type them.
- Green boxes = tips, yellow boxes = things to remember, red boxes = common traps.
- `$` at the start of a line only means 'your terminal prompt'. Do not type the dollar sign.

> **TIP:** Docker commands look the same on Windows, Mac and Linux. The only differences are small things like how you write file paths - I point them out.

Mini map of the journey

Chapter	You will learn
1-2	What Docker is and the words you must know
3-4	Install Docker and run your first containers
5-6	Manage containers and images with commands
7	Package your own app with a Dockerfile
8-10	Keep data (volumes), connect containers (networks), run a database
11-12	Docker Compose (many containers at once) and Docker Hub
13-15	Cleanup, troubleshooting, cheat sheet, practice

1. What is Docker?

You build an app on your laptop. It works. You send it to a friend (or a server) and it breaks: wrong Python version, missing library, different settings. Everybody has said **'but it works on my machine!'**

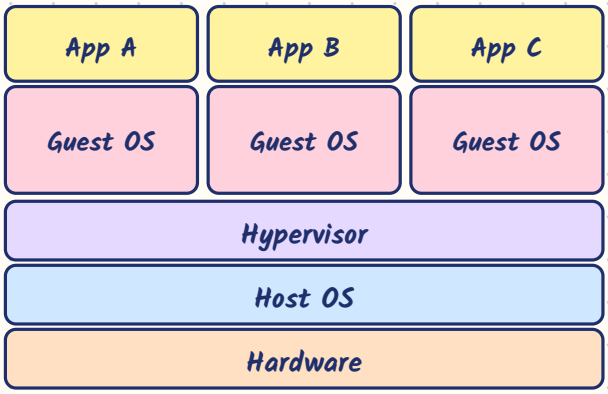
Docker fixes this by packing your app together with everything it needs (code, runtime, libraries, settings) into one sealed box called a **container**. That box runs the same way on any computer that has Docker.

REMEMBER: Think of a **shipping container**. It does not matter what is inside (bananas or laptops): ships, trains and trucks all handle it the same way. Docker does that for software.

Containers vs Virtual Machines

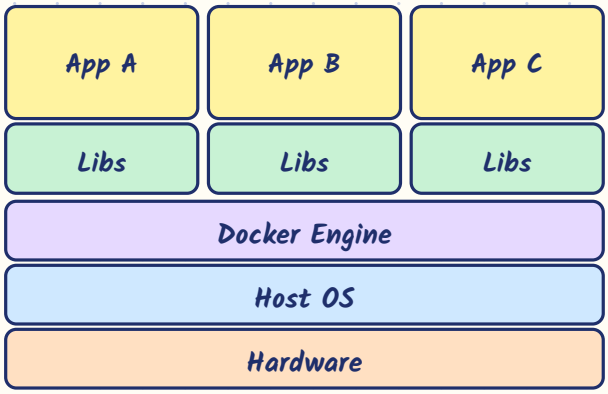
Virtual Machines

heavy: GBs, minutes to boot



Docker Containers

light: MBs, seconds to start



	Virtual Machine	Container
Size	Gigabytes (whole OS inside)	Megabytes (just app + libraries)
Start time	Minutes	Seconds or less
Isolation	Very strong (own OS)	Strong (shares host kernel)
How many on a laptop	A few	Dozens

Why people love Docker

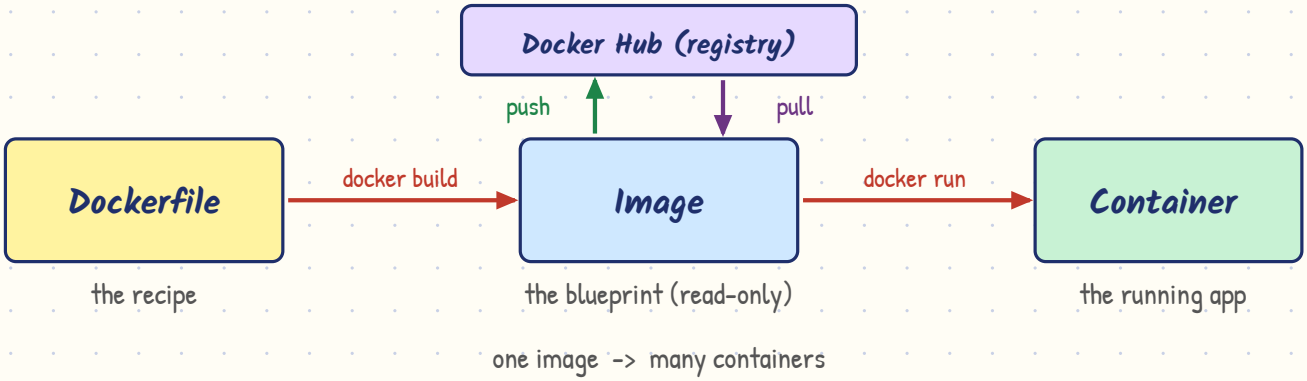
- **Same everywhere:** dev laptop, test server and production behave identically.
- **Fast setup:** new teammate runs one command instead of a 2-day install guide.
- **Isolated:** two apps needing different Python versions never fight each other.
- **Easy cleanup:** delete the container and the mess is gone.

2. The Vocabulary You Must Know

Word	Meaning in plain English
Image	A read-only **blueprint / template** that contains your app and everything it needs. Does not run by itself.
Container	A **running (or stopped) instance** of an image. You can start many containers from one image.
Dockerfile	A plain text file with step-by-step instructions to **build an image**.
Registry	A store for images. **Docker Hub** is the default public one (like GitHub for images).
Docker Engine	The program on your computer that builds and runs containers (the 'daemon' + the **docker** command).
Volume	A storage area outside the container so **data survives** when the container is deleted.
Network	A virtual network that lets containers **talk to each other**.

Word	Meaning in plain English
Compose	A tool + YAML file to **start several containers together** with one command.
Tag	A label on an image version, like python:3.12 (name:tag).

The big picture



REMEMBER: **Image = class / recipe. Container = object / cooked dish.** One recipe can make many dishes.

3. Installing Docker

Windows and Mac

- Download **Docker Desktop** from the official site: **docs.docker.com/get-docker**.
- Install it, open it, and wait until it says Docker is running (whale icon in the taskbar / menu bar).
- On Windows it uses WSL 2 - the installer guides you if you need to enable it.

Linux

Follow the install guide for your distro (Ubuntu, Debian, Fedora...) at **docs.docker.com/engine/install**. After installing, add yourself to the docker group so you do not need **sudo** every time:

```
sudo usermod -aG docker $USER
# log out and log in again for this to take effect
```

Check that it works

~ type these in your terminal

```
docker --version          # prints the Docker version
docker info               # details about your Docker setup
docker run hello-world    # downloads a tiny image and runs it
```

If **hello-world** prints 'Hello from Docker!' then everything works. Here is what just happened behind the scenes:

- 1. The **docker** client asked the Docker engine to run hello-world.
- 2. The engine did not find the image on your computer, so it **pulled** it from Docker Hub.

- 3. It created a **container** from that image and ran it.
- 4. The program printed its message and exited, so the container **stopped**.

4. Your First Containers

Example 1: talk to a Linux machine in seconds

```
docker run -it ubuntu bash
# you are now INSIDE an Ubuntu container. Try:
cat /etc/os-release       # shows Ubuntu info
ls /                      # list folders
exit                      # leave (the container stops)
```

`-i` keeps input open and `-t` gives you a terminal, so `-it` means interactive. `bash` is the command to run inside the container.

Example 2: run a real web server (nginx)

```
docker run -d -p 8080:80 --name my-web nginx
```

Now open **http://localhost:8080** in your browser - you will see the 'Welcome to nginx!' page. You just ran a web server without installing anything.

Understanding that command

Part	Meaning
`docker run`	create a container from an image and start it
`-d`	**detached**: run in the background (you get your terminal back)
`-p 8080:80`	**port mapping** host:container. Your port 8080 goes to port 80 inside the container
`--name my-web`	give the container a friendly name (otherwise Docker invents one)
`nginx`	the image to use. Pulled from Docker Hub if you do not have it

Look around, then clean up

```
docker ps                 # list RUNNING containers
docker logs my-web        # see what the server printed
docker stop my-web        # stop it
docker rm my-web          # delete it
```

More useful docker run flags

Flag	What it does
`--rm`	automatically delete the container when it stops (great for experiments)
`-e NAME=value`	set an **environment variable** inside the container
`-v host:container`	mount a folder or volume (see chapter 8)
`--network name`	attach the container to a network (see chapter 9)
`--restart unless-stopped`	restart the container automatically after crashes or reboot
`-p 3000:3000`	you can repeat -p for several ports

TIP: Pattern to remember: **docker run [flags] IMAGE [command]**. Flags always come before the image name.

5. Container Commands

A container goes through a simple life: **created -> running -> stopped -> removed**. These commands move it along:

Command	What it does
`docker ps`	list **running** containers
`docker ps -a`	list **all** containers (including stopped ones)
`docker run IMAGE`	create + start a new container
`docker start NAME`	start a stopped container again
`docker stop NAME`	stop politely (gives the app time to shut down)
`docker restart NAME`	stop and start
`docker kill NAME`	stop immediately (force)
`docker rm NAME`	delete a stopped container
`docker rm -f NAME`	force delete even if running
`docker logs NAME`	show output of the container
`docker logs -f NAME`	follow logs live (Ctrl+C to quit)
`docker exec -it NAME bash`	open a terminal **inside a running** container
`docker inspect NAME`	detailed info as JSON (IP address, mounts...)
`docker stats`	live CPU / memory usage
`docker top NAME`	processes running inside the container
`docker cp file NAME:/path`	copy a file into a container (reverse to copy out)

Example: run, enter, and break things safely

```
docker run -d --name box ubuntu sleep 3600   # keeps running for 1 hour
docker exec -it box bash                      # step inside
touch hello.txt && ls                         # make a file
exit
docker rm -f box                              # container + file gone
```

REMEMBER: **run** makes a new container every time. **start** revives an existing one. Beginners mix these up a lot!

TIP: You can use the first few characters of a container ID instead of the full name. If the ID is 3f4a9c... then **docker stop 3f4** works.

6. Image Commands

Command	What it does
`docker images`	list images stored on your computer
`docker pull nginx`	download an image (latest tag)
`docker pull python:3.12-slim`	download a specific version (tag)
`docker search redis`	search Docker Hub from the terminal
`docker rmi IMAGE`	remove an image
`docker history IMAGE`	show the layers an image is made of
`docker build -t name:tag .`	build an image from a Dockerfile in this folder
`docker tag SRC NEW`	give an image another name/tag
`docker push user/name:tag`	upload an image to Docker Hub
`docker login`	log in to Docker Hub

Image names and tags

An image name looks like `username/repository:tag`. Official images have no username, e.g. `nginx`, `python:3.12`, `node:20-alpine`.

- `alpine` / `slim` in a tag means a **smaller** version of the image.
- If you give no tag Docker uses `latest`.

> **CAREFUL!** **latest** does not mean 'newest and safest' - it is just a default name. In real projects pin a version like **python:3.12-slim** so builds stay predictable.

Layers (why builds are fast)

An image is a stack of **layers**, one per Dockerfile instruction. Docker **caches** layers: if a step and everything before it did not change, Docker reuses the cached layer instead of redoing the work. This idea is why the order of lines in a Dockerfile matters (next chapter).

7. Writing a Dockerfile

A **Dockerfile** is a text file (named exactly **Dockerfile**, no extension) that tells Docker how to build your image.

Main instructions

Instruction	Meaning
`FROM`	the **base image** to start from (always the first line)
`WORKDIR /app`	set the working folder inside the image (creates it if needed)
`COPY src dest`	copy files from your computer into the image
`RUN command`	run a command **while building** (install packages etc.)
`ENV KEY=value`	set an environment variable
`EXPOSE 3000`	document which port the app listens on (does not publish it)
`CMD ["a", "b"]`	the default command to run **when the container starts**
`ENTRYPOINT`	fixed command; CMD then supplies default arguments to it
`ARG`	a variable that exists only at build time

REMEMBER: **RUN** happens once when you build the image. **CMD** happens every time a container starts.

Project A: Node.js full-stack app (backend + frontend)

A small **to-do app**: an Express backend serves a JSON API and the frontend web page, all inside one container. Create a folder **node-todo** with this structure:

~ project layout

```
node-todo/
  package.json
  server.js            <- backend
  public/              <- frontend
    index.html
    style.css
    app.js
  Dockerfile
  .dockerignore
```

~ package.json

```
{
  "name": "node-todo",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": { "start": "node server.js" },
  "dependencies": { "express": "^4.19.2" }
}
```

~ server.js (backend)

```
const express = require("express");
const path = require("path");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));   // serves the frontend

let tasks = [{ id: 1, text: "Learn Docker", done: false }];

app.get("/api/tasks", (req, res) => res.json(tasks));

app.post("/api/tasks", (req, res) => {
  const text = (req.body.text || "").trim();
  if (!text) return res.status(400).json({ error: "text is required" });
  const task = { id: Date.now(), text, done: false };
  tasks.push(task);
  res.status(201).json(task);
});

app.delete("/api/tasks/:id", (req, res) => {
  tasks = tasks.filter((t) => t.id !== Number(req.params.id));
  res.status(204).end();
});

app.listen(PORT, "0.0.0.0", () => console.log("Server running on port " + PORT));
```

~ public/index.html (frontend)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Docker To-Do</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="card">
    <h1>My Docker To-Do</h1>
    <form id="form">
      <input id="text" placeholder="Add a task..." autocomplete="off">
      <button type="submit">Add</button>
    </form>
    <ul id="list"></ul>
  </div>
  <script src="app.js"></script>
</body>
</html>
```

~ public/style.css

```
body { font-family: Arial, sans-serif; background: #eef3ff; margin: 0; }
.card { max-width: 420px; margin: 60px auto; padding: 24px; background: white;
        border-radius: 12px; box-shadow: 0 4px 14px rgba(0,0,0,0.12); }
h1 { color: #1a5fb4; margin-top: 0; }
form { display: flex; gap: 8px; }
input { flex: 1; padding: 8px; }
button { padding: 8px 14px; cursor: pointer; }
li { display: flex; justify-content: space-between; padding: 8px 0;
     border-bottom: 1px solid #ddd; }
```

~ public/app.js (frontend logic)

```
const list = document.getElementById("list");
const form = document.getElementById("form");
const input = document.getElementById("text");

async function loadTasks() {
  const res = await fetch("/api/tasks");
  const tasks = await res.json();
  list.innerHTML = "";
  tasks.forEach((t) => {
    const li = document.createElement("li");
    li.textContent = t.text;
    const btn = document.createElement("button");
    btn.textContent = "Delete";
    btn.onclick = async () => {
      await fetch("/api/tasks/" + t.id, { method: "DELETE" });
      loadTasks();
    };
    li.appendChild(btn);
    list.appendChild(li);
  });
}

form.addEventListener("submit", async (e) => {
  e.preventDefault();
  await fetch("/api/tasks", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ text: input.value }),
  });
  input.value = "";
  loadTasks();
});

loadTasks();
```

~ Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

~ .dockerignore

```
node_modules
npm-debug.log
.git
.env
```

Line by line

`FROM node:20-alpine`	*small base image that already has Node.js 20*
`WORKDIR /app`	*all next commands run inside /app*
`COPY package*.json ./`	*copy ONLY the dependency files first (for caching!)*
`RUN npm install ...`	*install Express; cached until package.json changes*
`COPY . .`	*now copy backend + frontend code*
`EXPOSE 3000`	*note: the app uses port 3000*
`CMD [...]`	*how to start the server*

Build and run it

```
docker build -t node-todo .                          # the dot = 'use this folder'
docker run -d -p 3000:3000 --name todo node-todo
# open http://localhost:3000 in your browser - add and delete tasks!
curl http://localhost:3000/api/tasks                 # test the backend API directly
docker logs todo
docker stop todo && docker rm todo
```

*Here the **browser** loads the frontend from the container, and the frontend's JavaScript calls the backend API in the same container. Our tasks live in memory, so they disappear when the container restarts - that is why databases and volumes exist (chapters 8 and 10).*

TIP: Why copy package.json first? Your code changes often but your libraries rarely do. With this order Docker reuses the cached 'npm install' layer and rebuilds in seconds.

*CAREFUL! Do not copy **node_modules** from your computer into the image. That is why it is in **.dockerignore** - the image installs its own.*

Project B: Python FastAPI app

***FastAPI** is a modern Python framework for building APIs, and it creates interactive documentation for free. Make a folder **fastapi-books** with three files:*

~ main.py

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="Book API")


class Book(BaseModel):
    id: int
    title: str
    author: str


books: list[Book] = []


@app.get("/")
def home():
    return {"message": "Hello from FastAPI in Docker!"}


@app.get("/books")
def list_books():
    return books


@app.post("/books", status_code=201)
def add_book(book: Book):
    books.append(book)
    return book


@app.get("/books/{book_id}")
def get_book(book_id: int):
    for book in books:
        if book.id == book_id:
            return book
    raise HTTPException(status_code=404, detail="Book not found")
```

~ requirements.txt

```
fastapi==0.115.0
uvicorn[standard]==0.30.6
```

~ Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

*Same recipe as the Node example, but the base image is Python and the start command is **uvicorn** (the server that runs FastAPI). `main:app` means file main.py, object app. `--host 0.0.0.0` makes it reachable from outside the container.*

```
docker build -t fastapi-books .
docker run -d -p 8000:8000 --name books fastapi-books

# open the auto-generated docs and try the endpoints there:
#   http://localhost:8000/docs

curl http://localhost:8000/
curl -X POST http://localhost:8000/books -H "Content-Type: application/json" \
     -d '{"id": 1, "title": "Docker Basics", "author": "Asha"}'
curl http://localhost:8000/books
```

TIP: *On Windows Command Prompt/PowerShell, quoting JSON in curl is awkward - just use the **/docs** page in your browser to POST a book instead.*

TIP: *The app must listen on **0.0.0.0** (not 127.0.0.1) or you cannot reach it from outside the container. This is the #1 beginner bug. Both examples above already do this.*

8. Volumes: Keeping Your Data

*Everything written inside a container is **lost when the container is removed**. For databases, uploads and logs we need storage that lives outside the container. Two ways:*

Type	Use it for	Example flag
Named volume	*Docker manages the storage for you. Best for databases and app data.*	*-v mydata:/data*
Bind mount	*You choose a folder from your computer. Best for live-editing code.*	*-v ./src:/app*

Demo: data that survives

```
docker volume create mydata

docker run -it --rm -v mydata:/data ubuntu bash
# inside the container:
echo "I will survive" > /data/note.txt
exit

# the container is gone, but the volume is not:
docker run --rm -v mydata:/data ubuntu cat /data/note.txt
```

Volume commands

`docker volume ls`	*list volumes*
`docker volume create NAME`	*create a volume*
`docker volume inspect NAME`	*show where/how it is stored*
`docker volume rm NAME`	*delete a volume (data is lost!)*

Bind mount example: edit a website live

```
# in a folder that contains an index.html
docker run -d -p 8080:80 --name site -v "$(pwd)":/usr/share/nginx/html:ro nginx
```

*Edit **index.html**, refresh the browser - changes appear instantly. The **:ro** at the end means read-only inside the container. On Windows PowerShell use `${PWD}` instead of `$(pwd)`.*

9. Networking

Port mapping (outside world -> container)

Containers are isolated, so a port inside is not reachable from your browser until you publish it with `-p HOST:CONTAINER`. Example: `-p 3000:80` means localhost:3000 reaches port 80 in the container.

Container to container

*Containers on the **same custom network** can reach each other using the **container name as hostname**. No IP addresses needed!*

```
docker network create mynet
docker run -d --name web --network mynet nginx
docker run -it --rm --network mynet alpine sh
# inside the alpine container:
wget -qO- http://web           # prints the nginx welcome page
exit
```

`docker network ls`	*list networks*
`docker network create NAME`	*create a network*
`docker network inspect NAME`	*see which containers are connected*
`docker network connect NET NAME`	*attach a running container*
`docker network rm NAME`	*delete a network*

REMEMBER: *On the default network names do not resolve. Create your own network (or use Docker Compose, which does it automatically).*

10. Example: Running a Database

Databases are a perfect Docker use case: no installing, no leftovers. Let's start PostgreSQL:

```
docker run -d --name db \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  -p 5432:5432 \
  postgres:16
```

- `-e POSTGRES_PASSWORD=secret`: *required setting passed as an environment variable.*
- `-v pgdata:...`: *a named volume so data survives if you delete the container.*
- `\\` *at the end of a line = 'the command continues on the next line' (use ^ on CMD, ` on PowerShell).*

Talk to it

```
docker exec -it db psql -U postgres
# inside psql:
CREATE TABLE users (id SERIAL, name TEXT);
INSERT INTO users (name) VALUES ('Asha');
SELECT * FROM users;
\q
```

Now run **docker rm -f db**, start the same command again and select from users: the data is still there thanks to the volume.

CAREFUL! Never use a simple password like **secret** outside of practice. Real projects keep passwords in **.env** files or secrets managers and never commit them to Git.

II. Docker Compose

Real apps have several parts (web app + database + cache). Starting each with a long **docker run** is painful. **Docker Compose** lets you describe everything in one file called **compose.yaml** and start it all with one command.

Upgrade our FastAPI app with a Redis visit counter

We reuse the **fastapi-books** folder. Replace two files and add a third.

~ main.py (replace)

```
from fastapi import FastAPI
import redis

app = FastAPI()
cache = redis.Redis(host="redis", port=6379, decode_responses=True)


@app.get("/")
def home():
    hits = cache.incr("hits")
    return {"message": "Hello Docker!", "visits": hits}
```

~ requirements.txt (replace)

```
fastapi==0.115.0
uvicorn[standard]==0.30.6
redis==5.0.8
```

~ compose.yaml (new file, same folder)

```
services:
  web:
    build: .
    ports:
      - "8000:8000"
    depends_on:
      - redis
  redis:
    image: redis:7-alpine
    volumes:
      - redisdata:/data

volumes:
  redisdata:
```

*Notice **host="redis"** in the code: Compose puts both services on one network, so the service name **is** the hostname.*

Command	What it does
`docker compose up`	*create and start everything (add -d for background)*
`docker compose up -d --build`	*rebuild images first, then start*
`docker compose ps`	*list services*
`docker compose logs -f`	*follow logs of all services*
`docker compose exec web bash`	*open a shell in the web service*
`docker compose stop`	*stop (keeps containers)*
`docker compose down`	*stop and remove containers + network*
`docker compose down -v`	*...and also delete volumes*

```
docker compose up -d --build
curl http://localhost:8000      # {"message":"Hello Docker!","visits":1}
curl http://localhost:8000      # visits: 2 ... the counter grows!
docker compose down
```

*TIP: Compose files use **indentation with spaces** (never tabs). Two wrong spaces = an error, so copy carefully.*

12. Sharing Images with Docker Hub

*Create a free account at **hub.docker.com**, then:*

```
docker login
docker tag node-todo yourname/node-todo:1.0
docker push yourname/node-todo:1.0

# anyone (or any server) can now run it:
docker pull yourname/node-todo:1.0
docker run -d -p 3000:3000 yourname/node-todo:1.0
```

*Replace **yourname** with your Docker Hub username. The image name must start with it to be pushed.*

13. Cleaning Up

Docker keeps old containers, images and volumes until you remove them, so disk space can fill up.

`docker system df`	see how much space Docker uses
`docker container prune`	remove all **stopped** containers
`docker image prune`	remove **dangling** (untagged) images
`docker image prune -a`	remove all images not used by a container
`docker volume prune`	remove volumes not used by any container
`docker system prune`	stopped containers + unused networks + dangling images

CAREFUL! Prune commands delete things permanently. **docker volume prune** can erase database data - read the confirmation prompt before pressing y.

```
# remove ALL containers (running and stopped) - use with care
docker rm -f $(docker ps -aq)
```

14. When Things Go Wrong

Problem	Fix
port is already allocated	Another program/container uses that host port. Change the left number: **-p 8081:80**, or stop the other one.
container name already in use	Remove the old one: **docker rm -f NAME**, or pick another --name.
container exits immediately	Check **docker logs NAME**. A container stops when its main process ends.
cannot connect / empty reply	App listens on 127.0.0.1. Make it listen on **0.0.0.0** and check your -p mapping.
permission denied (Linux)	Add your user to the docker group (chapter 3) or use **sudo**.
pull access denied / not found	Typo in the image name or tag, or you are not logged in for a private image.
Cannot connect to the Docker daemon	Docker Desktop is not running. Start it and try again.
changes to code not showing	Rebuild: **docker build** again (or **compose up --build**), or use a bind mount.

TIP: First two debugging moves, always: **docker ps -a** (is it running?) and **docker logs NAME** (what did it say?).

15. Cheat Sheet

`docker run -d -p 80:80 IMG`	start in background
`docker ps / ps -a`	running / all
`docker stop / start NAME`	stop / start
`docker rm NAME`	delete container
`docker logs -f NAME`	live logs
`docker exec -it NAME bash`	shell inside
`docker images`	list images
`docker pull IMG`	download image
`docker rmi IMG`	delete image

`docker build -t n:t .`	build image
`docker tag / push / login`	share image
`docker volume create V`	make volume
`docker network create N`	make network
`docker compose up -d`	start stack
`docker compose down`	stop stack
`docker system prune`	cleanup
`docker stats`	resource usage
`docker inspect NAME`	all details

Practice tasks (tick them off!)

- [ ] Run **hello-world** and explain the 4 steps to a friend.
- [ ] Start nginx on port 8081, change its page with a bind mount, then stop and remove it.
- [ ] Build the Node.js to-do app, run it, add a task in the browser, then change the page title and rebuild.
- [ ] Build the FastAPI image and add a book using the /docs page.
- [ ] Start PostgreSQL with a volume, add a row, delete the container, recreate it, check the row.
- [ ] Create the FastAPI + Redis Compose project and watch the counter survive **docker compose restart**.
- [ ] Push your image to Docker Hub and pull it with a different tag name.

Where to go next

- **Multi-stage builds** to make production images smaller.
- **Health checks** and restart policies for reliable services.
- **Docker Compose profiles** and **.env** files for configuration.
- **Kubernetes** when you need to run containers across many servers.
- Official docs and tutorials: **docs.docker.com**

You made it! Keep experimenting – containers are cheap, so break things and delete them.